



Architectures et Administration des BDD

Présenté par:

Amal HALFAOUI (Epse GHERNAOUT)

Amal.halfaoui@univ-tlemcen.dz

amal.halfaoui@gmail.com

Université Tlemcen

-Ing 3-

- PL/SQL -

- Les curseurs -

Curseurs

- Les curseurs en PL/SQL sont des structures qui permettent de **parcourir des enregistrements retournés par une requête SQL**.
- Ils se divisent en deux types :
 - 1- implicites**
 - 2- explicites.**

Curseurs implicites

- Géré automatiquement par PL/SQL pour chaque opération du LMD de SQL (INSERT, UPDATE et DELETE).
- Pour le **SELECT** il faudra que le retour soit une seule ligne
- Ce curseur porte le nom **SQL**
- Les attributs de curseurs:
 - **SQL%ROWCOUNT** : Nombre de lignes affectées par la dernière Instruction LMD.
 - **SQL%FOUND** : TRUE si la dernière Instruction LMD affecte au moins un enregistrement
 - **SQL%NOTFOUND**: TRUE si la dernière Instruction LMD n'affecte aucun enregistrement

Curseurs implicites

Exemple

```
BEGIN
    DELETE FROM Pilote WHERE compa = 'AF';
    dbms_output.put_line('Le nombre de pilotes
AF supprimés est: ' || SQL%ROWCOUNT);
END;
```

Curseurs explicites

- Déclarés et manipulés par l'utilisateur
- Pour les interrogations qui retournent **plusieurs enregistrements**
- C'est une zone mémoire qui permet de traiter individuellement **chaque ligne renvoyée** par un SELECT.

Fonctionnement du curseur explicite

Ensemble de la requête

Brevet	NOM	NbHVol	COMPA
PL_1	HASSAINE	2700	AF
PL_2	SOUTOU	5400	AF
PL_3	BROWN	3000	SING

Pointeur curseur

Table Pilote

BREVET	NOM	NBHVOL	COMPA
PL-1	HASSAINE	2700	AF
PL-2	SOUTOU	5400	AF
PL-3	BROWN	3000	SING
PL-5	CHALABI	234	AF

Variable

NON

OUI

DECLARE

OPEN

FETCH

VIDE ?

Close

Créer une zone
mémoire SQL
EXP (Select * from
Pilote where
NbHVol>1000)

Chargement
des lignes
(L'ensemble
actif)

Se positionner
sur la ligne en
cours dans la
variable

Tester
l'existence des
lignes

Libérer la zone
mémoire

Fonctionnement du curseur explicite

```
DECLARE
  V_Pilote ...
  CURSOR C1 IS select Brevet,
                    Nom from Pilote where
                    COMPA = 'AF';

BEGIN
  .....
  OPEN C1;
  FETCH C1 INTO V_Pilote;
  .....
END
```

C1

Brevet	NOM	NbHVol	COMPA
PL_1	HASSAINE	2700	AF
PL_2	SOUTOU	5400	AF

PL_1	HASSAINE	2700	AF
------	----------	------	----

Instructions

- **Déclaration**

```
CORSOR nomcurseur IS requête;
```

- **Ouverture**

```
OPEN nomcurseur;
```

- **Utilisation**

```
FETCH nomcurseur INTO listevariables |  
    nomRECORD;
```

- **Fermeture**

```
CLOSE nomcurseur;
```

- **Attributs:** nomcurseur **%ISOPEN**, nomcurseur**%FOUND**,
 nomcurseur**%ROWCOUNT**, nomcurseur**%NOTFOUND**,

Parcours d'un curseur

- Suivant le traitement à effectuer sur le curseur à parcourir, choisir une structure répétitive **WHILE**, **LOOP** ou **FOR**
- Avant la première extraction, `nomCurseur%NOTFOUND` renvoie toujours **NULL**.
- Si l'instruction FETCH ne parvient jamais à s'exécuter correctement, la boucle devient infinie. Il est conseillé de programmer la sortie d'une structure répétitive à l'aide de la condition composée:
 - **EXIT WHEN** `nomCurseur%NOTFOUND` **OR**
 - `nomCurseur %NOTFOUND` **IS NULL**

Parcours d'un curseur (2)

Exemple WHILE

```
DECLARE
CURSOR C1 IS SELECT brevet , nom FROM Pilote where compa= 'AF' ;
v_brevet Pilote.brevet%TYPE;
v_nom Pilote.nom%TYPE;
BEGIN
OPEN C1;
FETCH C1 INTO v_brevet , v_nom ;
WHILE C1%FOUND LOOP
DBMS_OUTPUT.PUT_LINE('nom : ' || v_nom || ' (' || v_brevet || ') .' ) ;
FETCH C1 INTO v_brevet , v_nom ;
END LOOP;
END;
```

Procédure PL/SQL terminée.

```
nom : CHALABI      ( PL-5  ) .
nom : HASSAINE     ( PL-1  ) .
nom : SOUTOU       ( PL-2  ) .
```

Parcours d'un curseur (3)

Exemple LOOP

```
DECLARE
CURSOR C1 IS SELECT brevet , nom FROM Pilote where compa= 'AF' ;
v_brevet Pilote.brevet%TYPE;
v_nom Pilote.nom%TYPE;
BEGIN
OPEN C1;
LOOP
FETCH C1 INTO v_brevet , v_nom ;
EXIT WHEN C1%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('nom : ' || v_nom || ' ( ' || v_brevet || ' ) .' );
END LOOP;
END;
```

Parcours d'un curseur (4)

FOR (gestion semi automatique qui facilite la programmation)

- Evite les directives **OPEN, FETCH et CLOSE**). La boucle s'arrête d'elle-même à la fin de l'extraction de la dernière ligne du curseur
- **Déclaration automatique** de variable de réception du curseur.
- **Déclaration du curseur directement à l'intérieur de l'instruction FOR** (Curseur temporaire)

Parcours d'un curseur (5)

Exemple FOR

```
DECLARE
CURSOR C1 IS SELECT brevet , nom FROM Pilote where compa= 'AF' ;
BEGIN
FOR enreg IN C1 LOOP
DBMS_OUTPUT.PUT_LINE('nom : ' || enreg.nom || ' ( ' || enreg.brevet || ') . ;
END LOOP;
END;
```

Exemple FOR : Curseur temporaire

```
BEGIN
FOR enreg IN (SELECT brevet , nom FROM Pilote where compa= 'AF')
LOOP
DBMS_OUTPUT.PUT_LINE('nom : ' || enreg.nom || ' ( ' || enreg.brevet || ') . ;
END LOOP;
END;
```

Variables de réception du curseur

1- Variable scalaire ou %Type

Exemple

```
DECLARE
CURSOR C1 IS SELECT brevet , nom FROM Pilote where compa= 'AF' ;
v_brevet Pilote.brevet%TYPE;
v_nom Pilote.nom%TYPE; -----ou v_nom varchar (20);
BEGIN
OPEN C1;
LOOP
FETCH C1 INTO v_brevet , v_nom ;
EXIT WHEN C1%NOTFOUND;
Traitement...
END LOOP;
END;
```


Variables de réception du curseur (2)

2- Variable %ROWTYPE

Exemple

```
DECLARE
CURSOR C1 IS SELECT brevet ,nom FROM Pilote where compa= 'AF' ;
enreg C1%ROWTYPE;
BEGIN
OPEN C1;
LOOP
FETCH C1 INTO enreg ;
EXIT WHEN C1%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('nom : ' || enreg.nom || ' ( ' || enreg.brevet
    || ') . ' ) ;
END LOOP;
END;
```

Variables de réception du curseur (3)

3-Variable du type IS RECORD

Exemple

```
DECLARE
CURSOR C1 IS SELECT brevet ,nom FROM Pilote where compa= 'AF';
TYPE REC_Pilote IS RECORD(
v_brevet Pilote.brevet%TYPE,
v_nom Pilote.nom%TYPE);
enreg REC_Pilote;
BEGIN
OPEN C1;
LOOP
FETCH C1 INTO enreg ;
EXIT WHEN C1%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('nom : ' || enreg.v_nom || ' ( ' ||
    enreg.v_brevet || ') .' );
END LOOP; END;
```

Variables de réception du curseur (4)

4- Variable du type IS TABLE

valable pour les curseurs qui renvoient un nombre **raisonnable** de lignes

```
DECLARE
TYPE nomPilotes_tytabs IS TABLE OF Pilote.nom%TYPE
    INDEX BY BINARY_INTEGER;
tab_nom_pilote nomPilotes_tytabs;
CURSOR C1 IS SELECT brevet, nom FROM Pilote WHERE compa = 'AF' ;
i NUMBER := 1;
BEGIN
FOR enreg IN C1 LOOP
tab_nom_pilote(i) := enreg.nom; i:= i+1 ;
END LOOP;
DBMS_OUTPUT. PUT_LINE ( '2ème pilote :' || tab_nom_pilote(2) ) ;
END;
```

BULK COLLECT et LIMIT

- Les options de **FETCH** **BULK COLLECT** et **LIMIT** permettent de traiter de grands volumes de données d'une table sans pour autant pénaliser la mémoire ou le cache
- Utiliser un tableau PL/SQL qu'on chargera par partie dans une boucle en limitant le nombre d'enregistrement tout en parcourant toute la table volumineuse

```
FETCH nom-curseur BULK COLLECT INTO Variable_Table  
LIMIT Variable_limit;
```

Rq: Eviter %NOTFOUND et %FOUND dans les structures EXIT et WHILE. Préférez la méthode **COUNT** pour tester la taille réelle du tableau chargé par le BULK COLLECT .

BULK COLLECT et LIMIT (2)

Exemple

```
DECLARE
CURSOR C1 IS SELECT * FROM Pilote;
TYPE Pilote_tytabs IS TABLE OF Pilote%ROWTYPE INDEX BY BINARY_INTEGER;
tab_pilote Pilote_tytabs;
c_limit CONSTANT PLS_INTEGER := 100;
BEGIN
OPEN C1;
LOOP
    FETCH C1 BULK COLLECT INTO tab_pilote LIMIT c_limit;
    FOR i IN 1 .. tab_pilote.COUNT
    LOOP
        -- traitement de chaque ligne---
        DBMS_OUTPUT.PUT_LINE ( 'pilote :' || tab_pilote(i).brevet ) ;
    END LOOP;
    EXIT WHEN tab_pilote.count < c_limit;
END LOOP;
close C1;
END;
```

Paramètre d'un curseur

- Un curseur peut posséder des paramètres d'entrée
- utile lorsqu'un même curseur doit être utilisé plusieurs fois sous des critères différents
- Il faudra en ce cas fermer le curseur s'il était déjà utilisé, avant de l'ouvrir à nouveau en lui passant des paramètres différents
- Le passage des paramètres peut se faire à l'ouverture du curseur (OPEN) ou dans la boucle FOR (si le curseur est utilisé en mode semi-automatique)

Paramètre d'un curseur (2)

Exemple

```
DECLARE

CURSOR C1(p_codecomp IN VARCHAR2) IS

    SELECT brevet, nom FROM Pilote WHERE compa = p_codecomp;

enreg2 C1%ROWTYPE;

BEGIN

    -- 1er chargement et parcours du curseur avec le paramètre 'SING'
    FOR enreg IN C1('SING') LOOP
        DBMS_OUTPUT.PUT_LINE('SING, nom : ' || enreg.nom || '(' ||
            enreg.brevet || ')');
    END LOOP;

    -- 2e chargement et parcours du curseur avec le paramètre 'AF'
    OPEN C1('AF'); -- Ouvrir le curseur pour le paramètre 'AF'
    LOOP
        FETCH C1 INTO enreg2;
        EXIT WHEN C1%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('AF, nom : ' || enreg2.nom || '(' ||
            enreg2.brevet || ')');
    END LOOP; CLOSE C1; -- Fermer le curseur après utilisation

END;
```

Curseur modifiable (FOR UPDATE et CURRENT OF)

- Permet de **verrouiller** les lignes d'une table interrogée par un curseur dans le but de la **mettre à jour**
- Utilisation de la clause **FOR UPDATE** dans la déclaration du curseur
- Les verrous sont libérés à la fin de la transaction.

Syntaxe

```
CORSOR nomcurseur IS requête  
FOR UPDATE [ OF [{nomTable|nomVue}] [, ...]  
    [NOWAIT|WAIT entier]
```


Curseur modifiable (FOR UPDATE et CURRENT OF)

- La directive **OF** permet de connaître les colonnes à verrouiller. Sans elle, toutes les colonnes issues de la requête seront verrouillées.
- **NOWAIT** précise de ne pas faire attendre le programme et de retourner un message d'erreur si les lignes demandées sont verrouillées par une autre session.
- **WAIT** spécifie le nombre de secondes à attendre au maximum avant que les lignes soient déverrouillées par une autre session.
- Sans NOWAIT et WAIT, Le programme attend que les lignes soient disponibles.
- **Attention**: Il n'est pas possible de déclarer un curseur **FOR UPDATE** en utilisant dans la requête les directives **DISTINCT** ou **GROUP BY**, un **opérateur ensembliste**, ou une **fonction d'agrégat**.

Curseur modifiable (FOR UPDATE et CURRENT OF)

- Une fois que les lignes renvoyés par le curseur soient verrouillées par un FOR UPDATE, La modification est faite au niveau de la ligne courante du curseur par La clause **WHERE CURRENT OF**
- La clause **WHERE CURRENT OF** , située au niveau de l'instruction de mise à jour (**UPDATE** ou **DELETE**), permet de référencer la ligne courante d'un curseur.

Syntaxe

```
UPDATE | DELETE ... WHERE CURRENT OF  
nom_curseur
```

Curseur modifiable (FOR UPDATE et CURRENT OF)

Exemple

```
DECLARE
CURSOR CModifiable IS SELECT * FROM Pilote FOR UPDATE OF nbHVol
NOWAIT;
BEGIN
FOR enreg IN CModifiable LOOP
IF enreg.compa = 'AF' THEN
UPDATE Pilote SET nbHVol = nbHVol*100 WHERE CURRENT OF CModifiable;
ELSIF enreg.compa = 'SING' THEN UPDATE Pilote SET nbHVol = nbHVol -
100 WHERE CURRENT OF CModifiable;
ELSE DELETE FROM Pilote WHERE CURRENT OF CModifiable;
END IF;
END LOOP;
COMMIT;
END;END;
```

- Les exceptions -

Gestion des exceptions

- Afin d'éviter qu'un programme s'arrête à la première erreur (Exp requête ne retournant aucune ligne)
- Une exception PL/SQL correspond à une condition d'erreur et est associée à un identificateur
- 2 types d'exception :
 - Exception prédéfinie par Oracle
 - Exception définie par le programmeur

Gestion des exceptions

Exemple

```
BEGIN
...
IF (...) THEN RAISE PILOTE_TROP_JEUNE;
SELECT ... INTO ... FROM ...;
....
EXCEPTION
    WHEN NO DATA FOUND THEN
        Instructions A
    WHEN ZERO DIVIE THEN
        Instructions B
    WHEN PILOTE_TROP_JEUNE THEN
        Instructions C
    WHEN OTHERS THEN
        Instructions D; --- optionnel
END;
```

Prédéfinie

Définie par le programmeur

Gestion des exceptions

- Quelques exceptions prédéfinies

Nom de l'exception	Niméro	Commentaires
NO_DATA_FOUND	ORA-01403	Requête ne retournant aucun résultat.
CASE_NOT_FOUND	ORA-06592	Aucun des choix de la structure CASE sans ELSE n'est effectué
CURSOR_ALREADY_OPEN	ORA-06511	Ouverture d'un curseur déjà ouvert.
DUP_VAL_ON_INDEX	ORA-00001	Insertion d'une ligne en doublon (clé primaire).
ROWTYPE_MISMATCH	ORA-06504	Incompatibilité de types entre une variable externe et une variable PL/SQL.
TOO_MANY_ROWS	ORA-01422	Requête retournant plusieurs lignes.

Exceptions définies par le programmeur

Syntaxe

```
DECLARE
```

```
  NomException Exception
```

```
BEGIN
```

```
  IF.. Then Raise  NomException
```

```
EXCEPTION
```

```
WHEN NomException THEN Traitement;
```

```
END;
```


Exceptions définies par le programmeur

Exemple

```
DECLARE
V_NBVOL Number;
PILOTE_TROP_JEUNE EXCEPTION;
BEGIN
SELECT NBHVOL INTO V_NBVOL FROM Pilote where Brevet = 'PL_1';
IF V_NBVOL < 200 THEN RAISE PILOTE_TROP_JEUNE;
END IF;
EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE( 'Pas de pilote ayant ce code de brevet' );
WHEN PILOTE_TROP_JEUNE THEN
    DBMS_OUTPUT.PUT_LINE ( 'le pilote manque d''expérience' );
END;
```

Procédure RAISE_APPLICATION_ERROR

- permet de définir ses propres messages et codes d'erreurs.
- évite le renvoi d'exceptions non traitées car le numéro d'erreur (inclus dans RAISE_APPLICATION_ERROR) sera communiqué à l'environnement appelant (l'application)

Syntaxe

```
RAISE_APPLICATION_ERROR(numéroErreur, message)
```

- numéroErreur: valeur définie par l'utilisateur pour l'exception, comprise entre -20 000 et -20 999.
- message: chaîne de caractères (max 2 048 octets) décrivant l'erreur

Procédure RAISE_APPLICATION_ERROR

Exemple

```
DECLARE
V_NBVOL Number;
BEGIN
SELECT NBHVOL INTO V_NBVOL  FROM Pilote where Brevet = 'PL-1';
IF V_NBVOL < 200 THEN RAISE_APPLICATION_ERROR (-20300, 'le pilote
    manque d''expérience');
END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE( 'Pas de pilote ayant
        ce code de brevet');
END;
```